模型预测控制学习笔记

一、理论推导

刚性单质量体动力学模型如(1)所示：

$$j\ddot{x} = t_e - b\dot{x} - t_l. \tag{1}$$

j 是系统惯量，b 是粘滞阻尼系数，t_l 是负载力。控制器的设计需求是根据给定初始状态 x_0 以及参考运动轨迹 x_{ref}，设计 t_e，使得 x 跟踪 x_{ref}。其中需要满足约束：$|t_e| \leq t_{max}$, $\dot{x} \leq v_{max}$。

将(1)写成状态方程的形式：

$$\begin{cases} \dot{x_1} = x_2 \\ \dot{x_2} = \dfrac{t_e}{j} - \dfrac{b}{j}x_2 - \dfrac{t_l}{j}. \end{cases} \tag{2}$$

其中 $x_1 = x$，$x_2 = \dot{x}$。令 $\mathbf{x} = [x_1, x_2]^T$，$u = \frac{T_e}{j}$, $d = -\frac{t_l}{j}$，$c = \frac{b}{j}$，则状态方程可以写成:

$$\dot{\mathbf{x}} = \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & -c \end{bmatrix}}_{\mathbf{A}} \mathbf{x} + \underbrace{\begin{bmatrix} 0 \\ 1 \end{bmatrix}}_{\mathbf{B}} (u + d). \tag{3}$$

利用前向欧拉法将(3)离散化可得

$$\mathbf{x}[k+1] = \mathbf{A}_d \mathbf{x}[k] + \mathbf{B}_d (u + d). \tag{4}$$

其中，$\mathbf{A}_d = \mathbf{I} + \mathbf{A}T_s$，$\mathbf{B}_d = \mathbf{B}T_s$，$T_s$ 为采样时间。

假设初值为 $\mathbf{x}[0] = [x_1[0], x_2[0]]$，$\hat{\mathbf{x}}[k]$ 为第 k 步的状态变量预测值。模型预测控制的精髓在于利用 $\mathbf{x}[0]$ 和 $u[0], u[1], \cdots, u[N-1]$，通过(4)迭代计算 $\hat{\mathbf{x}}[1], \hat{\mathbf{x}}[2], \cdots, \hat{\mathbf{x}}[N]$，其中 N 为预测步长。然后，通过优化算法求解最优控制序列。

$$\begin{aligned} \hat{\mathbf{x}}[1] &= \mathbf{A}_d \mathbf{x}[0] + \mathbf{B}_d (u[0] + w[0]) \\ \hat{\mathbf{x}}[2] &= \mathbf{A}_d \hat{\mathbf{x}}[1] + \mathbf{B}_d (u[1] + w[1]) \\ &= \mathbf{A}_d^2 \mathbf{x}[0] + \mathbf{A}_d \mathbf{B}_d (u[0] + w[0]) + \mathbf{B}_d (u[1] + w[1]) \\ &\cdots \\ \hat{\mathbf{x}}[N] &= \mathbf{A}_d^N \mathbf{x}[0] + \sum_{i=0}^{N-1} \mathbf{A}_d^i \mathbf{B}_d (u[N-1-i] + w[N-1-i]) \end{aligned} \tag{5}$$

将(5)写成矩阵形式：

$$\hat{\mathbf{X}} = \mathbf{M}_x \mathbf{x}[0] + \mathbf{M}_{ud} (\mathbf{U} + \mathbf{W}). \tag{6}$$

其中，$\hat{\mathbf{X}} = [\hat{\mathbf{x}}[1], \hat{\mathbf{x}}[2], \cdots, \hat{\mathbf{x}}[N]]^T, \hat{\mathbf{X}} \in \mathbb{R}_{2N\times 1}$; $\mathbf{U} = [u[0], u[1], \cdots, u[N-1]]^T, \mathbf{U} \in \mathbb{R}_{N\times 1}$; $\mathbf{W} = [w[0], w[1], \cdots, w[N-1]]^T, \mathbf{W} \in \mathbb{R}_{N\times 1}$;

$$\mathbf{M}_x = [\mathbf{A}_d, \mathbf{A}_d^2, \cdots, \mathbf{A}_d^N]^T, \mathbf{M}_x \in \mathbb{R}_{2N\times 2}, \tag{7}$$

$$\mathbf{M}_{ud}=\begin{bmatrix}\mathbf{B}_d & 0 & \cdots & 0\\ \mathbf{A}_d\mathbf{B}_d & \mathbf{B}_d & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ \mathbf{A}_d^{N-1}\mathbf{B}_d & \mathbf{A}_d^{N-2}\mathbf{B}_d & \cdots & \mathbf{B}_d\end{bmatrix},\mathbf{M}_u\in\mathbb{R}_{2N\times N}, \tag{8}$$

目标函数设计为：

$$J(\mathbf{U})=\sum_{k=0}^{N-1}\|\mathbf{x}[k+1]-\mathbf{x}_r[k+1]\|_{\mathbf{Q}}^2+\sum_{k=0}^{N-1}\|u[k]\|_{\mathbf{R}}^2+\sum_{k=1}^{N-2}\|u[k+1]-u[k]\|_{\mathbf{S}}^2, \tag{9}$$

其中，$\mathbf{X}_r$ 是参考轨迹，$\mathbf{X}_r\in\mathbb{R}_{2N\times 2}$，$\mathbf{x}_r$ 是其中的一个参考轨迹点；$\mathbf{Q}\in\mathbb{R}_{2\times 2}$，$\mathbf{R}\in\mathbb{R}$，$\mathbf{S}\in\mathbb{R}$。

$$J(\mathbf{U})=\left(\hat{\mathbf{X}}-\mathbf{X}_r\right)^T\underbrace{\begin{bmatrix}\mathbf{Q} & 0 & \cdots & 0\\ 0 & \mathbf{Q} & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & \mathbf{Q}\end{bmatrix}}_{\bar{\mathbf{Q}}_{2N\times 2N}}\left(\hat{\mathbf{X}}-\mathbf{X}_r\right)+\mathbf{U}^T\underbrace{\begin{bmatrix}\mathbf{R} & 0 & \cdots & 0\\ 0 & \mathbf{R} & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & \mathbf{R}\end{bmatrix}}_{\bar{\mathbf{R}}_{\mathbf{N}\times\mathbf{N}}}\mathbf{U}+\mathbf{U}^T\mathbf{D}^T\begin{bmatrix}\mathbf{S} & 0 & \cdots & 0\\ 0 & \mathbf{S} & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & \mathbf{S}\end{bmatrix}\mathbf{D}\mathbf{U}. \tag{10}$$

其中，$\mathbf{D}$ 是差分矩阵，$\mathbf{D}\in\mathbb{R}_{N-1\times N}$，其定义为

$$\mathbf{D}=\begin{bmatrix}1 & -1 & 0 & \cdots & 0 & 0\\ 0 & 1 & -1 & \cdots & 0 & 0\\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots\\ 0 & 0 & 0 & \cdots & 1 & -1\end{bmatrix}. \tag{11}$$

将(11)代入(10)，化简后可得

$$J(\mathbf{U})=\left(\hat{\mathbf{X}}-\mathbf{X}_r\right)^T\underbrace{\begin{bmatrix}\mathbf{Q} & 0 & \cdots & 0\\ 0 & \mathbf{Q} & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & \mathbf{Q}\end{bmatrix}}_{\bar{\mathbf{Q}}_{2N\times 2N}}\left(\hat{\mathbf{X}}-\mathbf{X}_r\right)+\mathbf{U}^T\underbrace{\begin{bmatrix}\mathbf{S}+\mathbf{R} & -\mathbf{S} & 0 & \cdots & 0\\ -\mathbf{S} & \mathbf{2S}+\mathbf{R} & -\mathbf{S} & \cdots & 0\\ \vdots & \ddots & \ddots & \ddots & \vdots\\ 0 & \cdots & -\mathbf{S} & \mathbf{2S}+\mathbf{R} & -\mathbf{S}\\ 0 & 0 & \cdots & -\mathbf{S} & \mathbf{S}+\mathbf{R}\end{bmatrix}}_{\bar{\mathbf{S}}_{N\times N}}\mathbf{U}. \tag{12}$$

当且仅当 $\mathbf{R}>\mathbf{S}\left(2\cos\frac{\pi}{N+1}-1\right)$ 时，$\bar{\mathbf{S}}$ 为正定矩阵。

$$J(\mathbf{U})=\left[\mathbf{M}_x\mathbf{x}[0]+\mathbf{M}_{ud}(\mathbf{U}+\mathbf{W})-\mathbf{X}_r\right]^T\bar{\mathbf{Q}}\left[\mathbf{M}_x\mathbf{x}[0]+\mathbf{M}_{ud}(\mathbf{U}+\mathbf{W})-\mathbf{X}_r\right]+\mathbf{U}^T\bar{\mathbf{S}}\mathbf{U}. \tag{13}$$

$$J(\mathbf{U})=(\mathbf{U}+\mathbf{W})^T\mathbf{H}(\mathbf{U}+\mathbf{W})+\mathbf{f}^T(\mathbf{U}+\mathbf{W})+\text{const}. \tag{14}$$

其中

$$\begin{cases} \mathbf{H} = \mathbf{M}_{ud}^T \bar{\mathbf{Q}} \mathbf{M}_{ud} + \bar{\mathbf{S}} \\ \mathbf{f} = 2\mathbf{M}_{ud}^T \bar{\mathbf{Q}} \left[\mathbf{M}_x \mathbf{x}[0] - \mathbf{X}_r \right] \\ \text{const} = \left[\mathbf{M}_x \mathbf{x}[0] - \mathbf{X}_r \right]^T \bar{\mathbf{Q}} \left[\mathbf{M}_x \mathbf{x}[0] - \mathbf{X}_r \right]. \end{cases} \tag{15}$$

于是将 MPC 问题转换为求解约束 QP 问题。先不考虑扰动，$\mathbf{W} = \mathbf{0}$，约束条件为：

$$\begin{aligned} &\begin{bmatrix} I_N \\ -I_N \end{bmatrix} \mathbf{U} < \begin{bmatrix} 1_N \\ 1_N \end{bmatrix} u_{max}, \\ &\mathbf{C}\mathbf{M}_{ud}\mathbf{U} < \mathbf{C}1_{2N} v_{max} - \mathbf{C}\mathbf{M}_x \mathbf{x}[0] \\ &-\mathbf{C}\mathbf{M}_{ud}\mathbf{U} < \mathbf{C}1_{2N} v_{max} + \mathbf{C}\mathbf{M}_x \mathbf{x}[0] \end{aligned} \tag{16}$$

其中，$\mathbf{C}$ 为 one-hot 矩阵，用于选取偶数行，其表达式为：

$$\mathbf{C} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & \cdots & 0 & 1 \end{bmatrix}, \mathbf{C} \in \mathbb{R}^{N \times 2N}. \tag{17}$$

二、MATLAB 仿真

2.1 构造 plant 的状态空间方程

```
matlab
%% state space
format long;
b = 0.0001;
j = 0.001;
Ts = 0.0001;

c = b / j;
A = [0 1; 0 -c];
B = [0; 1];
C = [1 0];
D = 0;

A_d = eye(2) + A * Ts;
B_d = B * Ts;
C_d = C;
D_d = D;

plant = ss(A_d, B_d, C_d, D_d, Ts);
```

2.2 构造 MPC 的 H 矩阵和 f 向量

```
matlab
```

```
%% construct M_x, M_ud, x_0, \bar Q, \bar S, H and f
N = 10; % horizon

M_x = [];
for i = 1:N
    M_x = [M_x; A_d^i];
end

% Construct M_ud as a block lower-triangular Toeplitz matrix
% M_ud = [B_d      0       ...  0
%         A_d*B_d  B_d     ...  0
%         ...      ...     ...  ...
%         A_d^(N-1)*B_d  A_d^(N-2)*B_d  ...  B_d]
M_ud = [];
for i = 1:N
    row = [];
    for j = 1:N
        if i >= j
            row = [row, A_d^(i-j) * B_d];
        else
            row = [row, zeros(2, 1)];
        end
    end
    M_ud = [M_ud; row];
end

% Construct x_0
x_0 = [0; 0];

% Construct \bar Q
Q = [10000000 10; 10 100];
Q_bar = kron(eye(N), Q);

% Construct \bar R
R = 1e-2;
R_bar = kron(eye(N), R);

% Construct \bar S
S = 1e-4;
assert(R > S * (cos(pi / (N + 1)) - 1));

% Construct a N-1 x N differential Matrix
D = zeros(N - 1, N);
for i = 1:N-1
    for j = 1:N
        if i == j
```

```
            D(i, j) = 1;
        elseif j == i + 1
            D(i, j) = -1;
        end
    end
end

S_bar = kron(D'*D, S) + R_bar;

% Construct X_r
X_r = ones(2 * N, 1);

% Construct H
H = M_ud' * Q_bar * M_ud + S_bar;
H = (H + H') / 2;
f = 2 * M_ud' * Q_bar * (M_x * x_0 - X_r);
```

2.3 构造约束条件

```
matlab
u_max = 500;
v_max = 10;

a_1 = [eye(N); -1 * eye(N)];
b_1 = [ones(N, 1); ones(N,1)] * u_max;

C = zeros(N, 2*N);
for i = 1:N
    for j = 1:2*N
        if j == 2*i
            C(i, j) = 1;
        end
    end
end

a_2 = C * M_ud;
b_2 = C * ones(2*N, 1) * v_max - C * M_x * x_0;

a_3 = -C * M_ud;
b_3 = C * ones(2*N, 1) * v_max + C * M_x * x_0;

a = [a_1; a_2; a_3];
b = [b_1; b_2; b_3];
```

2.4 求解约束 QP 问题，并通过 quadprog 滚动优化得到最优控制序列

```
matlab
%% solve constrain QP
mode = 'step';
disturb = false;

step_num = 3e4;
time = 0:1:step_num-1+10;
time = time * Ts;

% 轨迹参数
P_end = 10;     % 终点位置 (m)
T_total = 5;    % 总运动时间 (s)

p1 = -2 * P_end / T_total^3;
p2 = 3 * P_end / T_total^2;

switch mode
    case 'poly'
        x_0 = [0; 0.5];
        x_r = p1 * time.^3 + p2 * time.^2;
        x_r_dot = 3 * p1 * time.^2 + 2 * p2 * time;
    case 'sin'
        x_0 = [0; 1 * pi];
        x_r = 1 * sin(2 * pi * time);
        x_r_dot = 2 * pi * cos(2 * pi * time);
    case 'step'
        x_0 = [0; 0];
        x_r = 1 * ones(1, length(time));   % step from 0 to 1 at t=0
        x_r_dot = zeros(1, length(time));  % zero velocity reference
    otherwise
        error('Unknown mode: %s', mode);
end

X_r = [x_r; x_r_dot];

options = optimoptions('quadprog', 'Display', 'off', ...
                       'Algorithm', 'interior-point-convex', ...
                       'MaxIterations', 100);

warm = [];

% 重塑参考轨迹为矩阵
X_r_mat = reshape(X_r, 2, []);   % 2行：位置，速度
```

```
% 预分配记录
X_record = zeros(length(x_0), step_num);
u_record = zeros(1,step_num);
u_rel_record = zeros(1,step_num);

% ---------- 离线计算 ----------
M_udQ = M_ud' * Q_bar;

% ---------- 在线循环 ----------
x_cur = x_0;
for i = 1:step_num
    % 提取当前及未来 N 步参考
    ref_window = X_r_mat(:, i:i+N-1);
    ref_vec = ref_window(:);   % 列向量

    % 计算梯度
    f = 2 * M_udQ * (M_x * x_cur - ref_vec);

    % 求解 QP
    [u, ~, exitflag] = quadprog(H, f, a, b, [], [], [], [], u, options);

    warm = u;   % 热启动

    % 系统更新
    if(disturb)
        x_next = A_d * x_cur + B_d * (u(1:1)  - 20 * sin(Ts * i * 2 * pi * 10));
    else
        x_next = A_d * x_cur + B_d * u(1:1);
    end
    X_record(:, i) = x_next;
    u_record(:, i) = u(1:1);
    if(disturb)
        u_rel_record(:,i) = u(1:1)  - 20 * sin(Ts * i * 2 * pi * 10);
    else
        u_rel_record(:,i) = u(1:1);
    end
    x_cur = x_next;
end
```

2.5 绘制结果

```
matlab
set(0, 'DefaultAxesFontName', 'Times New Roman');
set(0, 'DefaultTextFontName', 'Times New Roman');

figure('Position', [100, 100, 800, 600]);
```

```
subplot(3, 1, 1);
plot(time(1:step_num), X_r(1, 1:step_num), 'b--', 'LineWidth', 1.5); hold on;
plot(time(1:step_num), X_record(1, :), 'r-', 'LineWidth', 1.5);
xlabel('Time (s)', 'FontName', 'Times New Roman');
ylabel('Position (m)', 'FontName', 'Times New Roman');
legend('Reference', 'MPC', 'FontName', 'Times New Roman');
title('Position Tracking', 'FontName', 'Times New Roman');
grid on;

subplot(3, 1, 2);
plot(time(1:step_num), X_r(2, 1:step_num), 'b--', 'LineWidth', 1.5); hold on;
plot(time(1:step_num), X_record(2, :), 'r-', 'LineWidth', 1.5);
xlabel('Time (s)', 'FontName', 'Times New Roman');
ylabel('Velocity (m/s)', 'FontName', 'Times New Roman');
legend('Reference', 'MPC', 'FontName', 'Times New Roman');
title('Velocity Tracking', 'FontName', 'Times New Roman');
grid on;

subplot(3, 1, 3);
plot(time(1:step_num), u_record, 'g-', 'LineWidth', 1.5); hold on;
plot(time(1:step_num), u_rel_record, 'm-', 'LineWidth', 1.5);
xlabel('Time (s)', 'FontName', 'Times New Roman');
ylabel('Control Input', 'FontName', 'Times New Roman');
legend('u (control)', 'u_{rel} (actual)', 'FontName', 'Times New Roman');
title('Control Input (u)', 'FontName', 'Times New Roman');
grid on;
```

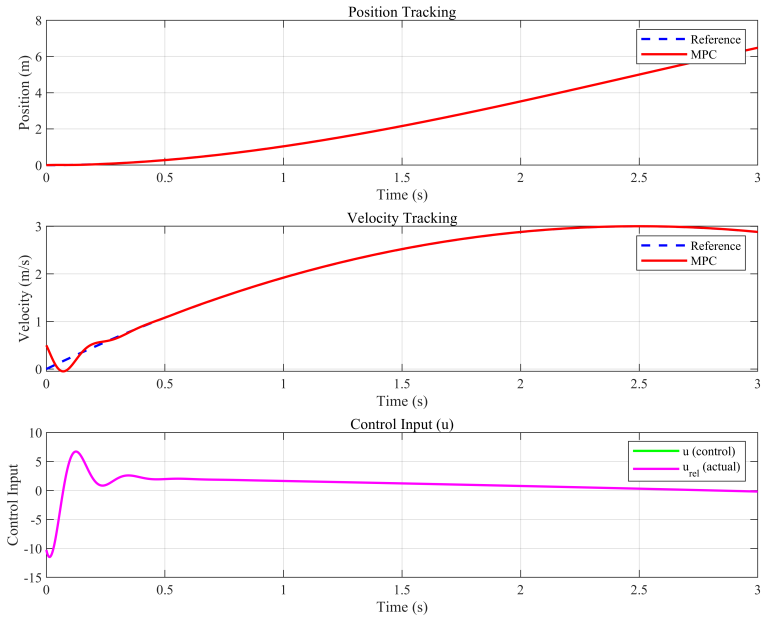

图 1: MPC 轨迹跟踪多项式轨迹仿真结果（无扰动）

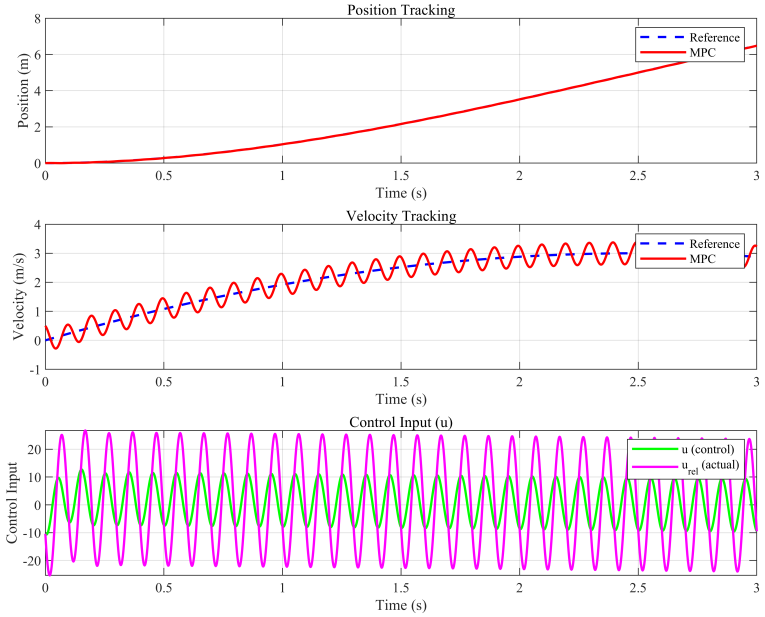

图 2: MPC 轨迹跟踪多项式轨迹仿真结果（正弦扰动）

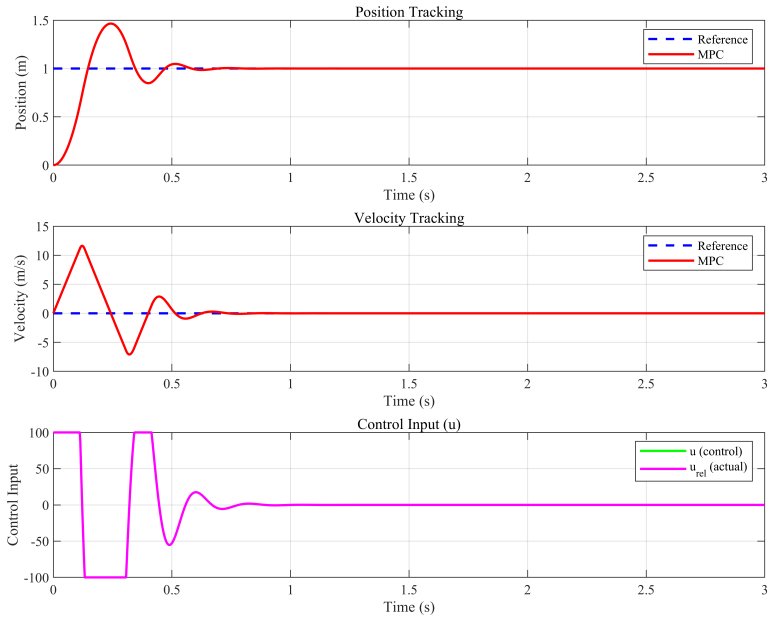

图 3: MPC 轨迹跟踪阶跃轨迹仿真结果（无扰动）

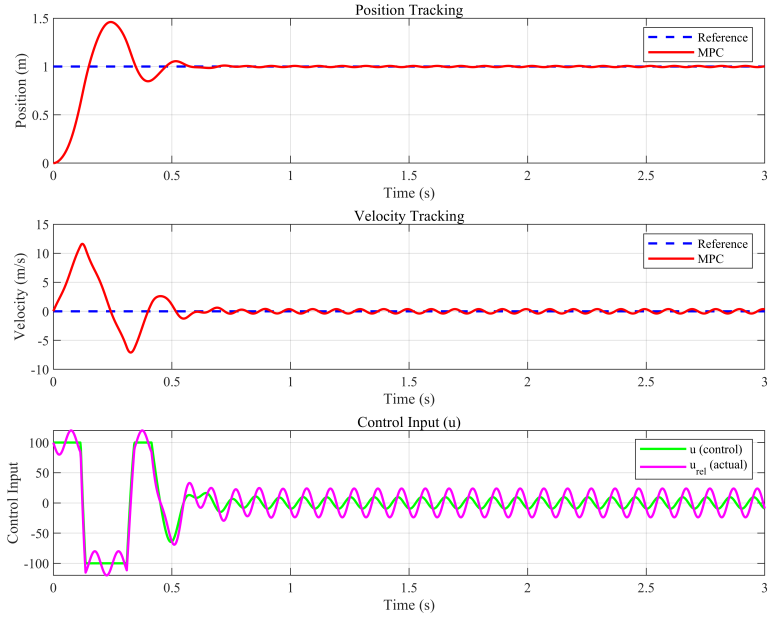

图 4: MPC 轨迹跟踪阶跃轨迹仿真结果（正弦扰动）

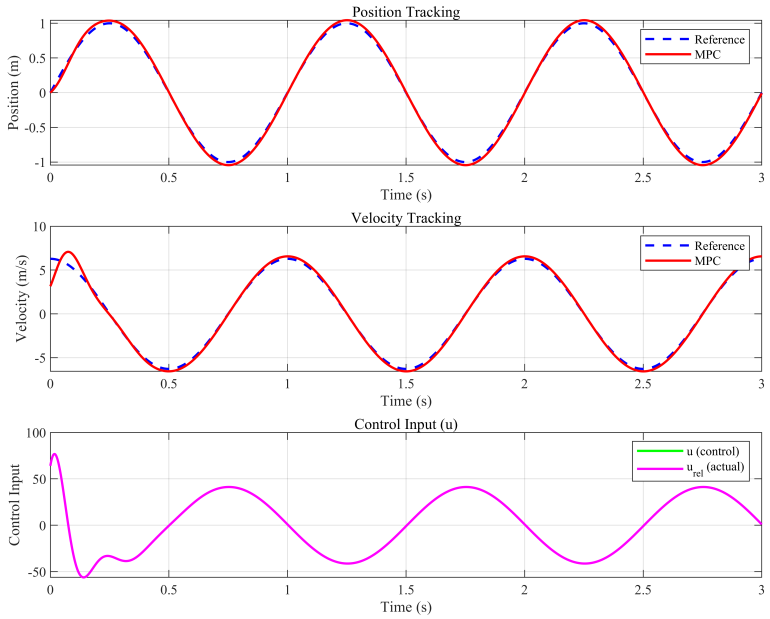

图 5: MPC 轨迹跟踪正弦轨迹仿真结果（无扰动）

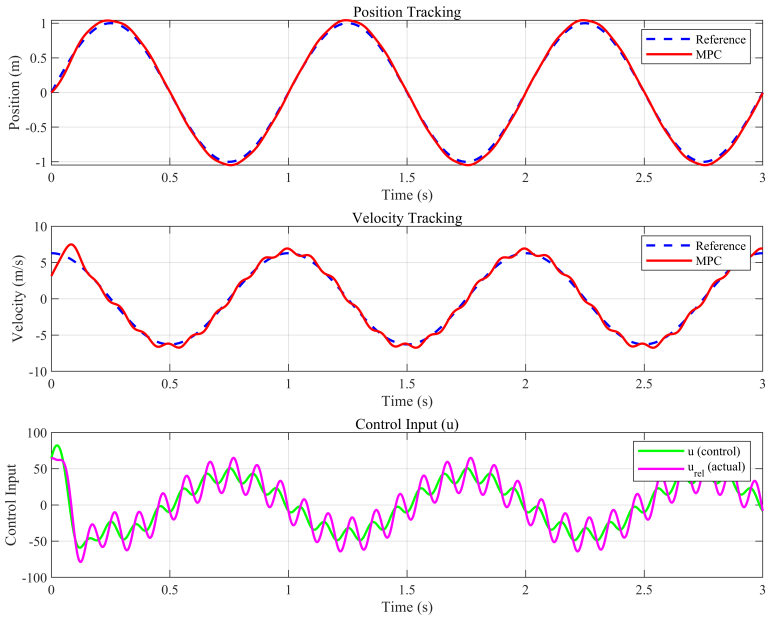

图 6: MPC 轨迹跟踪正弦轨迹仿真结果（正弦扰动）